

E-Commerce Database & Product Management API Design

Prepared from submitted ERD

August 22, 2026

Contents

1. Domain Grouping	2
2. Functional Requirements Coverage	3
2.1 Account & Identity	3
2.2 Product Catalog	4
2.3 Shopping & Fulfilment	6
2.4 Engagement	7
3. Entity-Relationship Diagram	9
4. Key Design Decisions	11
5. Handling Edge Cases	11
6. Scalability	12
7. Strong Consistency	13
8. REST API — Product Management	14

1. Domain Grouping

The original ERD groups into six functional domains:

Domain	Entities
Identity	site_user, user_address, address, country
Payment	user_payment_method, payment_type
Catalog (product management — primary focus)	product_category, product, product_item, variation, variation_option, product_configuration, promotion, promotion_category
Cart	shopping_cart, shopping_cart_item
Order	shop_order, order_line, order_status, shipping_method
Feedback	user_review

2. Functional Requirements Coverage

This section walks through the site's functional requirements one by one and explains what each means in practice and exactly which entities, columns, and constraints in the schema satisfy it.

2.1 Account & Identity

Requirement	What it means	How the schema satisfies it
Create Accounts	Users can create accounts and log in.	<code>site_user</code> is the authentication anchor: <code>email_address</code> is <code>UNIQUE</code> (used as the login identifier) and <code>password_hash</code> stores a salted hash, never plaintext.
Contact Details	Users store their name, email, and phone number.	<code>site_user.first_name</code> , <code>site_user.last_name</code> , <code>site_user.email_address</code> , and <code>site_user.phone_number</code> together cover full contact info for the account.

Requirement	What it means	How the schema satisfies it
Addresses	Users can store one or more addresses and mark one as the default shipping address.	address holds the reusable postal record; user_address is the many-to-many join between site_user and address, carrying is_default. A partial unique index (uq_user_default_address, WHERE is_default = true) guarantees at most one default address per user at the database level, not just in application code.
Payment Methods	Users can store one or more payment methods and mark one as the default.	user_payment_method (tokenized account_number, expiry_date) with the same is_default + partial-unique-index pattern as addresses (uq_user_default_payment) — exactly one default per user, enforced by the database.

2.2 Product Catalog

Requirement	What it means	How the schema satisfies it
Products	The site stores a large number of products.	product is a lean row (name, description, image URL, category, active flag) indexed on category_id and on name (trigram index) so search and browse stay fast at catalog scale — see §6 Scalability.
Categories	Every product belongs to a category; categories can belong to other categories.	product_category is self-referencing via parent_category_id, forming the category tree. product.category_id is a required FK. A CHECK constraint plus a trigger (see §5 Edge Cases) stop a category from becoming its own ancestor.
Product Variations	A product can have variations (e.g. colour, size), and each variation has its own values (e.g. red, blue).	variation models the variation type (e.g. “Color”) scoped to a category; variation_option models its values (e.g. “Red”, “Blue”). product_configuration is the junction that attaches a specific combination of option values to a specific product_item (SKU) — e.g. SKU #123 = {Color: Red, Size: XL}.

Requirement	What it means	How the schema satisfies it
Number in Stock	The site tracks how many of each product are in stock.	product_item.qty_in_stock, constrained CHECK (qty_in_stock >= 0) so it can never go negative. Stock is tracked per SKU (product_item), not per product, since stock genuinely varies by variation (e.g. Red/S vs. Red/XL). Concurrency-safe decrementing is covered in §7 Strong Consistency.

2.3 Shopping & Fulfilment

Requirement	What it means	How the schema satisfies it
Shopping Cart	Visitors can add products to a cart; carts are only saved in the database once the visitor is logged in.	shopping_cart.user_id is NOT NULL and UNIQUE — a cart row can only exist for an authenticated user. Guest browsing keeps cart contents client-side (session storage); a database row is created only at login/sign-up, at which point the client-side cart is written into shopping_cart_item.

Requirement	What it means	How the schema satisfies it
Payment Details for Order	Placing an order requires payment and address details.	<code>shop_order.payment_method_id</code> (FK → <code>user_payment_method</code>) and <code>shop_order.shipping_address</code> (FK → <code>address</code>) are captured at order time, so every order is tied to the exact payment method and address used.
Shipping Method	The user picks a shipping method from a list; each method has one standard price.	<code>shipping_method(name, price)</code> is a simple lookup table; <code>shop_order.shipping_method</code> references it, and <code>shipping_method.price</code> is a flat rate per method (CHECK (price >= 0)), matching the “single standard price” requirement.
Order Status	Orders move through stages such as processing, delivery in progress, and delivered.	<code>order_status</code> is a lookup table of stage names; <code>shop_order.order_status</code> (FK) holds the order’s current stage. Modelling status as a foreign key (rather than a free-text column) means the valid stage list is centrally managed and query-able.

2.4 Engagement

Requirement	What it means	How the schema satisfies it
Reviews	Users can review products they've purchased, with a 1-5 rating and a text comment.	<code>user_review.rating_value</code> is CHECK (<code>rating_value</code> BETWEEN 1 AND 5); <code>comment</code> is free text. Critically, <code>ordered_product_id</code> is a FK to <code>order_line</code> (not <code>product</code>), so a review can only be attached to a line item the user actually bought — see §5 Edge Cases.
Promotions	The site can run promotions giving one or more product categories a specific discount.	<code>promotion(discount_rate, start_date, end_date)</code> defines the sale; <code>promotion_category</code> is the junction letting one promotion cover multiple categories (and one category carry multiple promotions), with <code>discount_rate</code> constrained to (0, 1].

3. Entity-Relationship Diagram

The diagram below redraws all entities and their relationships (replacing the original schema listing), grouped by domain, with primary keys (**PK**), foreign keys (**FK**), and cardinality on every relationship (1 = exactly one, N = many, 0..1 = zero-or-one).

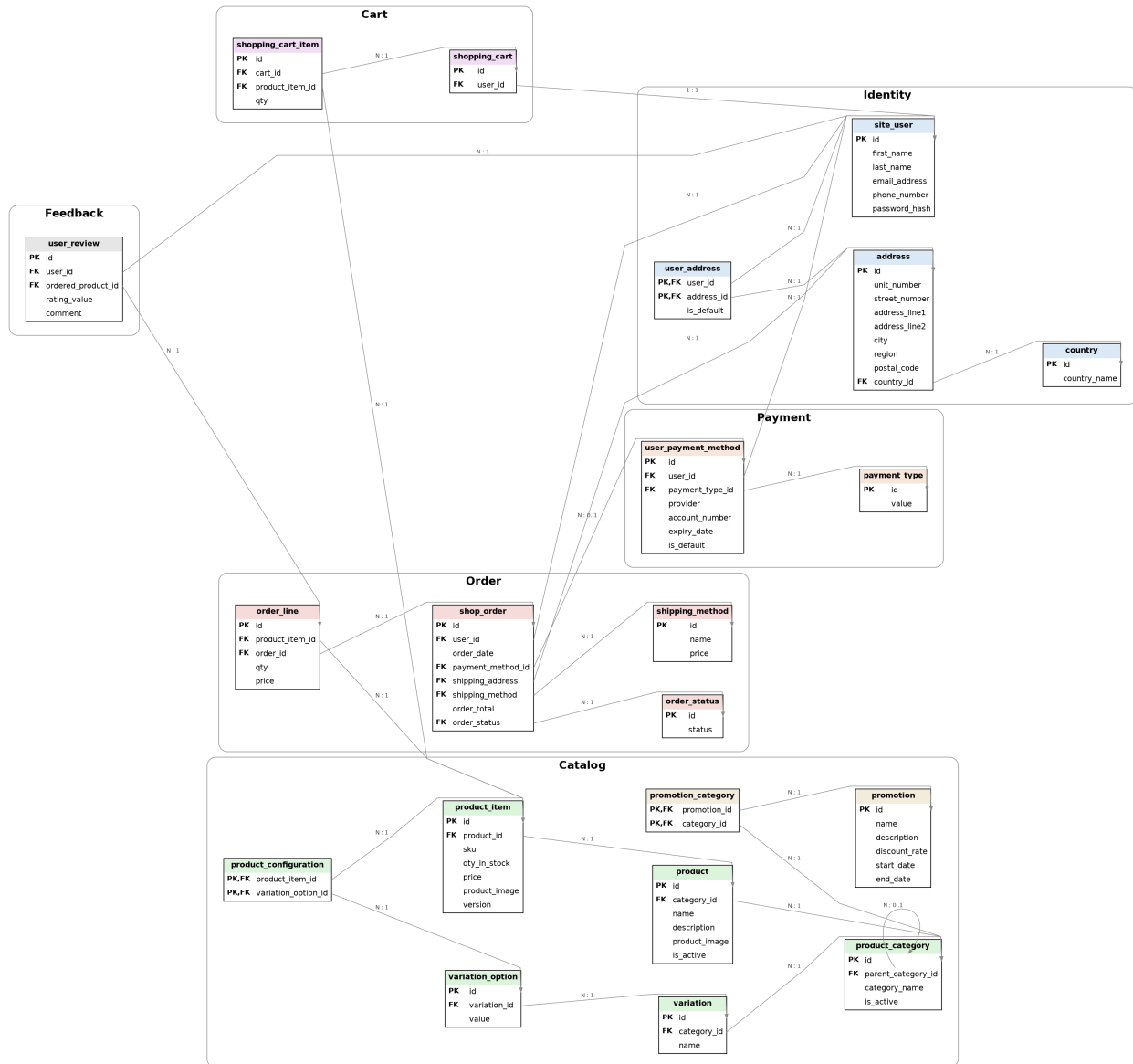


Figure 1: Entity-Relationship Diagram

Notable relationships:

- product_category.parent_category_id → product_category.id — self-referencing (category tree).

- `product_item.product_id` → `product.id` — a product has many purchasable SKU/variants.
- `product_configuration` — junction table resolving the many-to-many between `product_item` and `variation_option` (e.g. SKU #123 = {Color: Red, Size: XL}).
- `promotion_category` — junction table resolving the many-to-many between `promotion` and `product_category`.
- `order_line.product_item_id` → `product_item.id` — order lines always reference the sold SKU, with price stored as a snapshot, independent of the SKU's current price.
- `user_review.ordered_product_id` → `order_line.id` — a review is tied to an actual purchased line item, not just a product.

4. Key Design Decisions

- **order_line.price is a snapshot**, decoupled from product_item.price. Changing a product's price later never rewrites order history.
- **product_image stores a URL**, not binary data — actual files live in object storage (S3/GCS) behind a CDN, keeping rows small and read latency low.
- **Soft delete via is_active** on product and product_item instead of hard DELETE, since products can be referenced by historical orders (ON DELETE RESTRICT on those foreign keys backs this up).
- **version column on product_item** enables optimistic concurrency control for admin edits and stock adjustments (detailed in §5).

5. Handling Edge Cases

Edge case	Mechanism
Overselling (two customers buy the last unit at once)	Atomic conditional update: <code>UPDATE product_item SET qty_in_stock = qty_in_stock - :qty WHERE id = :id AND qty_in_stock >= :qty</code> , check affected-row count — never “read then write” in separate steps.
Concurrent admin edits (two staff editing the same SKU)	Optimistic locking via version: update succeeds only <code>WHERE id = :id AND version = :expected</code> . 0 rows affected → 409 Conflict.
Category cycles (A → B → A)	<code>CHECK (id <> parent_category_id)</code> blocks the trivial case; a BEFORE INSERT/UPDATE trigger walks the ancestor chain to block deeper cycles.
Duplicate variant combination (two SKUs both = {Red, XL})	Unique index on a hash of the sorted variation_option_id set per product_id, enforced in the same transaction that inserts product_configuration rows.

Edge case	Mechanism
Two options of the same variation on one item	Trigger on product_configuration insert rejects a new option if that item already has an option from the same variation_id.
Product with no variations	Always create exactly one default product_item per product at creation time.
Deleting a category with active products	ON DELETE RESTRICT blocks it; API returns 409 and requires reassigning products first.
Overlapping promotions on one category	Either resolve “highest discount wins” at query time, or use EXCLUDE USING gist on (category_id, tstzrange(start_date, end_date)) to physically prevent overlap.
Review without a purchase	FK to order_line (not product) plus UNIQUE(user_id, ordered_product_id) — a review requires an actual purchased line item, once each.
Multi-currency / regional pricing	Not modeled in the base ERD — extend with a price_list(product_item_id, currency_code, price) table rather than overloading product_item.price.

6. Scalability

- **Read/write split:** catalog reads (browsing, search) vastly outnumber writes — route GET traffic to read replicas, keep writes (price, stock, catalog edits) on the primary.
- **Caching:** cache the product_category tree and hot products in Redis with short TTL and explicit invalidation on write. Stock is never served from cache at check-out time — always re-verified against the primary.

- **Search:** mirror product/product_item into Elasticsearch/OpenSearch via CDC (e.g. Debezium) for browse/search endpoints; Postgres stays canonical for consistency.
- **Pagination:** keyset/cursor pagination (WHERE id > :cursor ORDER BY id LIMIT :n) instead of OFFSET, which degrades at scale.
- **Partitioning:** shop_order / order_line can be range-partitioned by order_date once volume grows, keeping catalog tables lean.
- **Static assets:** product images served from a CDN, never from the app tier.

7. Strong Consistency

- PostgreSQL provides ACID by default; the two places that need explicit protection are **inventory decrement** and **catalog edits**:
 - *Inventory:* a single conditional UPDATE (or SELECT ... FOR UPDATE inside a transaction) guarantees no oversell under concurrent load.
 - *Catalog edits:* optimistic concurrency via version avoids long lock durations while still preventing silent overwrites.
- Multi-table writes that must succeed or fail together (e.g. creating a product_item plus its product_configuration rows) are wrapped in a single database transaction.
- Foreign keys are enforced by the database in the catalog domain — not left to application-level checks alone.

8. REST API — Product Management

Base path: /api/v1

Method	Path	Purpose
GET	/categories	List categories (tree or flat, ?parent_id=)
POST	/categories	Create category
PATCH	/categories/{id}	Update category (rename, re-parent)
DELETE	/categories/{id}	Delete (409 if it has products or child categories)
GET	/products?category_id=&q=&cursor=&limit=	List/search products (cursor pagination)
POST	/products	Create product + default product_item
GET	/products/{id}	Product detail incl. items and variations
PATCH	/products/{id}	Update name/description/category
DELETE	/products/{id}	Soft delete (is_active = false)
GET	/products/{id}/items	List SKUs/variants for a product
POST	/products/{id}/items	Add a new SKU/variant with variation options
PATCH	/product-items/{id}	Update price/image; requires If-Match: <version>

Method	Path	Purpose
POST	/product-items/{id}/inventory/adjust	Atomic stock delta, idempotent via Idempotency-Key
GET	/categories/{id}/variations	List variations and options for a category
POST	/variations, /variations/{id}/options	Manage variation taxonomy
GET / POST	/promotions	Manage promotions
POST	/promotions/{id}/categories/{categoryId}	Attach promotion to a category

Concurrency control on writes: PATCH /product-items/{id} requires the client to send the last-known version via If-Match. The server executes UPDATE ... WHERE id = :id AND version = :v and returns 409 Conflict with the current state on mismatch.

Idempotency: inventory-mutating endpoints accept an Idempotency-Key header; the server stores recently-seen keys (e.g. Redis, 24h TTL) and replays the prior response instead of re-applying the delta, protecting against retries during network blips.

Example — atomic stock adjustment:

```
UPDATE product_item
SET qty_in_stock = qty_in_stock + :delta,
    version      = version + 1,
    updated_at   = now()
WHERE id = :id
  AND qty_in_stock + :delta >= 0
RETURNING qty_in_stock;
-- 0 rows returned => 409 Conflict (insufficient stock)
```

Errors: standard problem-detail JSON — 400 validation, 404 not found, 409 conflict (stock or version), 422 business rule violation (e.g. cyclic category).